

# Đối tượng dữ liệu tuần tự

Trịnh Tấn Đạt

Đại Học Sài Gòn

[trinhtandat@sgu.edu.vn](mailto:trinhtandat@sgu.edu.vn)

<http://sites.google.com/site/ttdat88>

# Nội dung

- Đối tượng dữ liệu iterable
- Một số hàm cơ bản trên lớp các đối tượng tuần tự
- Đối tượng iterator
- Đối tượng generator

# Mục tiêu học tập

Sau khi hoàn thành chương này, sinh viên có thể:

- Phân biệt sequence, iterable, iterator và generator.
- Sử dụng for để duyệt dữ liệu đúng cách.
- Thao tác trên list, tuple, string và range.
- Sử dụng các hàm dựng sẵn để xử lý dữ liệu tuần tự.
- Tạo iterator và generator đơn giản.
- Lựa chọn list hay generator trong bài toán có dữ liệu lớn.
- Viết chương trình xử lý dòng dữ liệu theo cách tiết kiệm bộ nhớ.

# Lưu đồ khái niệm

Sequence



Iterable



Iterator



Từng phần tử dữ liệu

Generator



Một cách đặc biệt để tạo Iterator



Sinh giá trị theo yêu cầu

Lưu ý:

- Không phải mọi iterable đều là sequence.
- Iterator luôn là iterable.
- Generator là một dạng iterator.
- List, tuple, string là các iterable phổ biến.

# Sequence là gì?

- Sequence là đối tượng dữ liệu có:
  - Thứ tự phần tử.
  - Vị trí chỉ mục, thường bắt đầu từ 0.
  - Khả năng truy cập phần tử bằng index.
  - Thường hỗ trợ slicing.
- Ví dụ sequence phổ biến:

```
numbers = [10, 20, 30]  
coordinates = (10.8, 106.6)  
name = "Python"  
values = range(1, 6)
```

# Iterable là gì?

- Iterable là đối tượng có thể được duyệt từng phần tử, thường bằng vòng lặp for.

```
values = [10, 20, 30]

for value in values:
    print(value)
```

- Các iterable phổ biến: list, tuple, str, range, set, dict, File object, Generator, Iterator

# for hoạt động như thế nào?

```
for value in values:  
    print(value)
```

về ý tưởng tương đương:

```
iterator = iter(values)  
  
while True:  
    try:  
        value = next(iterator)  
        print(value)  
    except StopIteration:  
        break
```

## Lưu ý:

- for gọi iter() để tạo iterator.
- for gọi next() từng lần.
- Khi không còn phần tử, iterator phát sinh StopIteration.
- for tự động bắt StopIteration và kết thúc.

# Kiểm tra iterable

```
from collections.abc import Iterable
```

```
values = [1, 2, 3]
```

```
number = 10
```

```
name = "Python"
```

```
print(isinstance(values, Iterable))
```

```
print(isinstance(number, Iterable))
```

```
print(isinstance(name, Iterable))
```

Kết quả:

True

False

True

## Lưu ý:

- Số nguyên là không iterable.
- Chuỗi là iterable vì có thể duyệt từng ký tự.
- Dictionary khi duyệt trực tiếp sẽ duyệt qua key.

# Duyệt dictionary

```
station = {  
    "id": "A01",  
    "name": "Tan Son Hoa",  
    "pm25": 32.5  
}
```

```
for key in station:  
    print(key)
```

```
for key, value in station.items():  
    print(f"{key}: {value}")
```

```
for value in station.values():  
    print(value)
```

# Ví dụ với dữ liệu quan trắc

```
pm25_values = [12.5, 18.2, 35.1, 28.4, 41.8]
```

```
for index, value in enumerate(pm25_values):  
    print(f"Lần đo {index}: {value}")
```

# Bài tập thực hành 1

## Bài 1. Nhận diện kiểu dữ liệu

Cho các đối tượng sau:

`a = [1, 2, 3]`

`b = (10, 20)`

`c = "Python"`

`d = 100`

`e = {"A01": 32.5}`

`f = range(5)`

Yêu cầu:

- Dự đoán đối tượng nào là iterable.
- Dùng `isinstance(obj, Iterable)` để kiểm tra.
- Dùng `for` để duyệt các đối tượng iterable.
- Giải thích kết quả khi duyệt dictionary.

# Bài tập thực hành 1

## Bài 2. Duyệt dữ liệu cảm biến

```
records = [  
    ("Ao1", 32.5),  
    ("Ao2", 18.2),  
    ("Ao3", 41.8)  
]
```

- In từng mã trạm và giá trị PM2.5.
- Đếm số trạm có PM2.5 lớn hơn 25.
- Tính tổng PM2.5 bằng vòng lặp.

# Các hàm cơ bản trên dữ liệu tuần tự

- Indexing và slicing

```
values = [10, 20, 30, 40, 50]
```

```
print(values[0])  
print(values[-1])
```

```
print(values[1:4])  
print(values[:3])  
print(values[::-2])  
print(values[::-1])
```

| Biểu thức    | Kết quả       |
|--------------|---------------|
| values[0]    | 10            |
| values[-1]   | 50            |
| values[1:4]  | [20, 30, 40]  |
| values[::-2] | [10, 30, 50]  |
| values[::-1] | Đảo ngược dãy |

# Các hàm dựng sẵn quan trọng

```
values = [12.5, 18.2, 35.1, 28.4, 41.8]
```

```
print(len(values))  
print(sum(values))  
print(min(values))  
print(max(values))  
print(sorted(values))  
print(sorted(values, reverse=True))
```

| Hàm        | Công dụng           |
|------------|---------------------|
| len()      | Số phần tử          |
| sum()      | Tổng các giá trị    |
| min()      | Giá trị nhỏ nhất    |
| max()      | Giá trị lớn nhất    |
| sorted()   | Tạo dãy đã sắp xếp  |
| reversed() | Duyệt ngược dữ liệu |

# enumerate()

```
stations = ["Tan Son Hoa", "Thu Duc", "Bien Hoa"]  
  
for index, station in enumerate(stations):  
    print(index, station)
```

```
stations = ["Tan Son Hoa", "Thu Duc", "Bien Hoa"]  
  
for index, station in enumerate(stations, start=1):  
    print(f"{index}. {station}")
```

## Khi nào dùng?

- Cần cả vị trí và giá trị.
- In danh sách có đánh số.
- Gắn chỉ số thời gian hoặc số thứ tự quan sát.

# zip()

```
stations = ["A01", "A02", "A03"]  
pm25_values = [32.5, 18.2, 41.8]  
  
for station, pm25 in zip(stations, pm25_values):  
    print(f"{station}: {pm25}")
```

Tạo dictionary:

```
station_data = dict(zip(stations, pm25_values))  
print(station_data)
```

## Lưu ý

- zip() dừng theo iterable ngắn nhất.
- Trong Python 3, zip() trả về iterator.

# any() và all()

```
pm25_values = [12.5, 18.2, 35.1, 28.4, 41.8]

print(any(value > 35 for value in pm25_values))
print(all(value >= 0 for value in pm25_values))
```

Ý nghĩa:

| Hàm           | Trả về True khi             |
|---------------|-----------------------------|
| any(iterable) | Có ít nhất một phần tử đúng |
| all(iterable) | Tất cả phần tử đều đúng     |

# map() và filter() với iterable

```
temperatures = [28.5, 30.0, 31.2, 29.8]

fahrenheit = map(lambda c: c * 9 / 5 + 32, temperatures)

hot_days = filter(lambda c: c > 30, temperatures)

print(list(fahrenheit))
print(list(hot_days))
```

## Lưu ý:

- map() và filter() trả về iterator.
- Muốn in hoặc dùng lại kết quả nhiều lần, có thể chuyển sang list.

# Bài tập thực hành 2: Phân tích chuỗi số

Bài 1: Cho dữ liệu:

`pm25 = [12.5, 18.2, 35.1, 28.4, 41.8, 20.6, 30.2]`

Yêu cầu:

1. In số phần tử, tổng, nhỏ nhất, lớn nhất và trung bình.
2. Tạo list mới chỉ gồm các giá trị tại vị trí chẵn.
3. Tạo list đảo ngược.
4. Sắp xếp giảm dần mà không làm thay đổi list gốc.
5. Kiểm tra có giá trị nào lớn hơn 40 không.
6. Kiểm tra tất cả giá trị có không âm không.

# Bài tập thực hành 2: Kết hợp dữ liệu

Bài 2: Cho dữ liệu:

```
station_ids = ["Ao1", "Ao2", "Ao3", "Ao4"]
```

```
station_names = ["Tan Son Hoa", "Thu Duc", "Bien Hoa", "Nha Be"]
```

```
pm25_values = [32.5, 18.2, 41.8, 25.0]
```

Yêu cầu:

1. Dùng zip() duyệt đồng thời ba danh sách.

2. In theo định dạng:

```
Ao1 - Tan Son Hoa: 32.5 µg/m3
```

3. Tạo list tuple: ("Ao1", "Tan Son Hoa", 32.5)

4. Tạo dictionary: {"Ao1": 32.5, "Ao2": 18.2, ...}

5. Dùng enumerate() để đánh số từng bản ghi bắt đầu từ 1.

# Iterator là gì?

- Iterator là đối tượng:
  - Có thể trả về phần tử tiếp theo bằng next().
  - Nhớ vị trí hiện tại trong quá trình duyệt.
  - Khi hết phần tử, phát sinh StopIteration.
  - Có thể dùng trong vòng lặp for.

```
values = [10, 20, 30]  
iterator = iter(values)
```

```
print(next(iterator))  
print(next(iterator))  
print(next(iterator))
```

10  
20  
30

# Iterable và iterator

| Đặc điểm                | Iterable            | Iterator                     |
|-------------------------|---------------------|------------------------------|
| Có thể dùng trong for   | Có                  | Có                           |
| Dùng iter()             | Trả về iterator     | Thường trả về chính nó       |
| Dùng next() trực tiếp   | Thường không        | Có                           |
| Có thể duyệt lại từ đầu | Thường có           | Không tự động                |
| Ví dụ                   | List, tuple, string | iter(list), generator, zip() |

```
values = [10, 20, 30]
```

```
print(iter(values))
```

```
print(iter(values) is values)
```

```
<list_iterator object at 0x787a6af88700>
```

```
False
```

# StopIteration

```
values = [10, 20]  
iterator = iter(values)
```

```
print(next(iterator))  
print(next(iterator))  
print(next(iterator))
```

10

20

Traceback (most recent call last):

File "/home/main.py", line 6, in <module>

print(next(iterator))

~~~~^

StopIteration

Lần gọi cuối phát sinh: StopIteration

Xử lý thủ công:

```
try:
```

```
    print(next(iterator))
```

```
except StopIteration:
```

```
    print("Không còn phần tử")
```

# Iterator chỉ dùng một lần

```
values = [10, 20, 30]  
iterator = iter(values)
```

```
print(list(iterator))  
print(list(iterator))
```

## Kết quả

```
[10, 20, 30]  
[]
```

## Giải thích

- Sau lần chuyển sang list đầu tiên, iterator đã đi đến cuối.
- Iterator không tự quay về đầu.
- Muốn duyệt lại, cần tạo iterator mới: `iterator = iter(values)`

# Ví dụ: Iterator từ zip()

```
stations = ["A01", "A02", "A03"]  
pm25 = [32.5, 18.2, 41.8]  
  
paired = zip(stations, pm25)  
  
print(next(paired))  
print(next(paired))  
  
print(list(paired))
```

**Kết quả**  
("A01", 32.5)  
("A02", 18.2)  
[("A03", 41.8)]

# Xây dựng iterator

- Một iterator do người dùng xây dựng thường cần:

`__iter__()`

`__next__()`

## Giải thích từng bước

1. for gọi `iter(counter)`.
2. `__iter__()` trả về chính counter.
3. for gọi `next(counter)`.
4. `__next__()` trả từng giá trị.
5. Khi vượt end, phát sinh `StopIteration`.

```
class CountUp:
    def __init__(self, start, end):
        self.current = start
        self.end = end

    def __iter__(self):
        return self

    def __next__(self):
        if self.current > self.end:
            raise StopIteration

        value = self.current
        self.current += 1
        return value
```

## Kết quả

1  
2  
3  
4  
5

```
counter = CountUp(1, 5)

for value in counter:
    print(value)
```

# Bài tập thực hành 3: iterator

- Bài tập 1: Cho: `values = [10, 20, 30, 40]`

Yêu cầu:

1. Tạo iterator từ `values`.
2. Gọi `next()` hai lần và in kết quả.
3. Chuyển phần còn lại của iterator thành list.
4. Thử gọi `next()` sau khi iterator hết phần tử.
5. Tạo iterator mới và duyệt lại toàn bộ dữ liệu.

# Bài tập thực hành 3: iterator

Bài 2: Tạo iterator dãy số chẵn

Viết class: `EvenNumbers(start, end)`

Yêu cầu:

- Duyệt các số chẵn từ start đến end.
- Nếu start là số lẻ, giá trị đầu tiên phải là số chẵn tiếp theo.
- Dùng được trong vòng lặp for.

Ví dụ:

```
for x in EvenNumbers(3, 12):  
    print(x)
```

Kết quả:

4  
6  
8  
10  
12

# Hint

```
class EvenNumbers:
    def __init__(self, start, end):
        self.current = start
        self.end = end

        if self.current % 2 != 0:
            self.current += 1

    def __iter__(self):
        return self

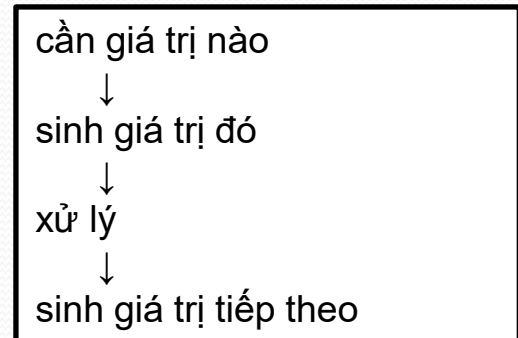
    def __next__(self):
        if self.current > self.end:
            raise StopIteration

        value = self.current
        self.current += 2
        return value
```

```
for value in EvenNumbers(5, 20):
    print(value)
```

# Generator là gì?

- Generator là một cách tạo iterator đơn giản bằng:
  - Hàm có từ khóa yield.
  - Generator expression.
- Generator:
  - Sinh giá trị theo yêu cầu.
  - Không tạo toàn bộ dữ liệu trong bộ nhớ ngay từ đầu.
  - Tự duy trì trạng thái giữa các lần gọi.
  - Tự phát sinh StopIteration khi hết dữ liệu.



# return và yield

| Đặc điểm           | return           | yield                   |
|--------------------|------------------|-------------------------|
| Kết thúc hàm       | Có               | Không ngay lập tức      |
| Trả dữ liệu        | Một lần          | Nhiều lần               |
| Lưu trạng thái hàm | Không            | Có                      |
| Kiểu hàm tạo ra    | Hàm thông thường | Generator function      |
| Khi gọi hàm        | Chạy ngay        | Trả về generator object |

```
def normal_function():  
    return 1
```

```
def generator_function():  
    yield 1
```

# Generator function cơ bản

```
def count_up_to(n):  
    current = 1  
  
    while current <= n:  
        yield current  
        current += 1
```

```
generator = count_up_to(5)  
  
print(next(generator))  
print(next(generator))  
  
for value in generator:  
    print(value)
```

## Kết quả

1  
2  
3  
4  
5

# Generator quan sát dữ liệu

```
def high_pm25_values(values, threshold=25):  
    for value in values:  
        if value > threshold:  
            yield value
```

```
pm25 = [12.5, 18.2, 35.1, 28.4, 41.8, 20.6]
```

```
for value in high_pm25_values(pm25):  
    print(value)
```

Kết quả  
35.1  
28.4  
41.8

# Generator expression

```
numbers = [1, 2, 3, 4, 5]

squares_list = [x ** 2 for x in numbers]

squares_generator = (x ** 2 for x in numbers)

print(squares_list)
print(squares_generator)
print(list(squares_generator))
```

Kết quả  
[1, 4, 9, 16, 25]  
<generator object <genexpr> at 0x7a4f15e9a9b0>  
[1, 4, 9, 16, 25]

| Kiểu                 | Cú pháp                           |
|----------------------|-----------------------------------|
| List comprehension   | [expression for item in iterable] |
| Generator expression | (expression for item in iterable) |

# List và generator

| Tiêu chí            | List            | Generator                     |
|---------------------|-----------------|-------------------------------|
| Tạo toàn bộ dữ liệu | Có              | Không                         |
| Bộ nhớ              | Nhiều hơn       | Ít hơn                        |
| Truy cập index      | Có              | Không                         |
| Duyệt lại           | Có              | Không tự động                 |
| Tốc độ tạo ban đầu  | Có thể chậm hơn | Thường nhanh hơn              |
| Phù hợp             | Dữ liệu nhỏ/vừa | Dữ liệu lớn hoặc dòng dữ liệu |

```
values = [x*x for x in range(1000000)]
```

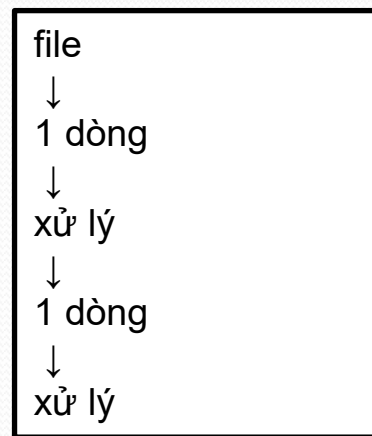
```
values = (x*x for x in range(1000000))
```

# Generator xử lý file lớn

```
def read_non_empty_lines(file_path):  
    with open(file_path, encoding="utf-8") as file:  
        for line in file:  
            line = line.strip()  
  
            if line:  
                yield line  
  
for line in read_non_empty_lines("data.txt"):  
    print(line)
```

## Ý nghĩa:

- Không đọc toàn bộ file vào RAM.
- Chỉ giữ một dòng tại mỗi thời điểm.
- Phù hợp với log file hoặc dữ liệu cảm biến lớn.



# Generator pipeline

```
def read_values(values):  
    for value in values:  
        yield value  
  
def valid_values(values):  
    for value in values:  
        if value is not None and value >= 0:  
            yield value  
  
def high_values(values, threshold):  
    for value in values:  
        if value > threshold:  
            yield value
```

```
raw = [12.5, -999, None, 18.2, 35.1, 41.8]  
  
pipeline = high_values(  
    valid_values(read_values(raw)),  
    threshold=25  
)  
  
print(list(pipeline))
```

```
raw  
↓  
read_values()  
↓  
valid_values()  
↓  
high_values()  
↓  
result
```

Kết quả  
[35.1, 41.8]

# Bài tập Thực hành 4: Generator cơ bản

- Bài 1. Generator số chia hết cho 3

Viết generator:

```
multiples_of_three(start, end)
```

Yêu cầu:

Sinh lần lượt các số chia hết cho 3 trong khoảng từ start đến end.

Không tạo list trung gian.

Ví dụ:

```
print(list(multiples_of_three(5, 20)))
```

Kết quả:

```
[6, 9, 12, 15, 18]
```

- Bài 2. Generator bình phương

Viết generator nhận list số và lần lượt sinh bình phương của mỗi giá trị.

# Bài tập Thực hành 4: Generator dữ liệu quan trắc

- Bài 3: Cho dữ liệu:

`raw_pm25 = [12.5, -999, None, 18.2, 35.1, -1, 41.8, 20.6, 30.2]`

Viết pipeline generator gồm:

`valid_pm25(values)`

`high_pm25(values, threshold)`

Yêu cầu:

`valid_pm25` chỉ sinh giá trị kiểu số và không âm.

`high_pm25` chỉ sinh giá trị lớn hơn `threshold`.

Dùng pipeline để lấy các giá trị PM2.5 hợp lệ và lớn hơn 25.

Tính tổng và trung bình mà không chuyển toàn bộ generator thành list ngay từ đầu.

# Hint

```
def valid_pm25(values):  
    for value in values:  
        if isinstance(value, (int, float)) and value >= 0:  
            yield value  
  
def high_pm25(values, threshold=25):  
    for value in values:  
        if value > threshold:  
            yield value
```

```
raw_pm25 = [12.5, -999, None, 18.2, 35.1, -1, 41.8, 20.6, 30.2]
```

```
pipeline = high_pm25(  
    valid_pm25(raw_pm25),  
    threshold=25  
)
```

```
total = 0  
count = 0
```

```
for value in pipeline:  
    total += value  
    count += 1
```

```
mean = total / count if count > 0 else None
```

```
print("Tổng:", total)  
print("Số giá trị:", count)  
print("Trung bình:", mean)
```

# Bài tập tổng hợp

**Bài toán:** Phân tích dữ liệu PM2.5 theo luồng

**Cho dữ liệu:**

```
raw_data = [  
    ("A01", 12.5),  
    ("A02", -999),  
    ("A03", 35.1),  
    ("A04", None),  
    ("A05", 41.8),  
    ("A06", 20.6),  
    ("A07", 30.2)  
]
```

**Yêu cầu:**

- Viết generator `valid_records(records)` chỉ sinh record có PM2.5 là số và không âm.
- Viết generator `high_pollution_records(records, threshold=25)`.
- Dùng `enumerate()` đánh số các record vượt ngưỡng.
- Dùng `zip()` ghép tên mức độ ô nhiễm với các ngưỡng tương ứng.
- Tính số record hợp lệ, số record vượt ngưỡng, tổng và trung bình PM2.5 của nhóm vượt ngưỡng.
- Không tạo list trung gian không cần thiết.

# Hint

```
def valid_records(records):
    for station_id, value in records:
        if isinstance(value, (int, float)) and value >= 0:
            yield station_id, value

def high_pollution_records(records, threshold=25):
    for station_id, value in records:
        if value > threshold:
            yield station_id, value
```

```
valid_stream = valid_records(raw_data)

high_stream = high_pollution_records(
    valid_stream,
    threshold=25
)

total = 0
count = 0

for index, (station_id, value) in enumerate(high_stream, start=1):
    print(f"{index}. {station_id}: {value}")
    total += value
    count += 1

mean = total / count if count else None
```

# Tổng kết Chương 3

- Iterable là đối tượng có thể duyệt từng phần tử.
- Sequence là iterable có thứ tự và thường có hỗ trợ index/slicing.
- Iterator theo dõi trạng thái duyệt và trả phần tử bằng next().
- for tự động sử dụng cơ chế iter() và next().
- Generator là một dạng iterator giúp sinh dữ liệu theo yêu cầu.
- yield cho phép tạm dừng và tiếp tục hàm tại lần gọi sau.
- Generator rất hữu ích cho pipeline xử lý dữ liệu và dữ liệu kích thước lớn.
- Kiến thức này là nền tảng để làm việc với NumPy, Pandas, file dữ liệu, chuỗi thời gian và pipeline khoa học dữ liệu ở các chương sau